

CSS @property Variables

Complete Guide: Easy English Edition

Master custom properties with type validation and default values in modern CSS

Contents

1	Introduction	2
1.1	What is @property?	2
1.2	Why Use @property?	2
2	Custom Properties Basics	3
2.1	What Are Custom Properties?	3
2.2	Traditional CSS Variables	3
3	@property Syntax	4
3.1	Basic Syntax	4
3.1.1	Three Required Descriptors	4
3.2	Supported Syntax Types	4
4	Color Properties	5
4.1	Color Property Example	5
4.2	Multiple Color Variables	5
5	Numeric Properties	6
5.1	Length Properties	6
5.2	Percentage Properties	6
5.3	Number Properties	7
6	Animated Properties	8
6.1	Animating Custom Properties	8
6.2	Color Animation	8
7	Real-World Examples	9
7.1	Example 1: Design System with @property	9
7.2	Example 2: Theme Switcher	10
7.3	Example 3: Animated Gradient	10
7.4	Example 4: Responsive Spacing	11
8	Advanced Techniques	12
8.1	Computed Values	12
8.2	Responsive Properties	12
8.3	Fallback Values	13
9	Browser Support	14
9.1	Compatibility	14
9.2	Progressive Enhancement	14
10	Common Patterns	15
10.1	Pattern 1: Design Tokens	15

10.2 Pattern 2: State Management	15
10.3 Pattern 3: Animations Library	15
11 Common Mistakes	17
11.1 Mistake 1: Missing Required Descriptors	17
11.2 Mistake 2: Wrong Initial Value Type	17
11.3 Mistake 3: Forgetting Inheritance	17
12 Quick Reference	19
12.1 @property Syntax	19
12.2 Common Properties	19
12.3 Key Points	19

1 Introduction

CSS `@property` is a modern feature that lets you define custom CSS properties with types, default values, and inheritance rules. It's like creating your own CSS properties with superpowers!

1.1 What is `@property`?

The `@property` at-rule lets you register custom properties with:

- **Type Validation:** Define what kind of value the property accepts
- **Default Values:** Set initial values
- **Inheritance:** Control whether values inherit to children
- **Animation Support:** Animate custom properties properly

1.2 Why Use `@property`?

- **Better Control:** Type-safe custom properties
- **Cleaner Code:** More readable CSS
- **Animation Ready:** Animate custom properties smoothly
- **Error Prevention:** Type checking catches mistakes
- **Documentation:** Your intent is clear
- **Development Tools:** Better IDE support

2 Custom Properties Basics

2.1 What Are Custom Properties?

Custom properties (CSS variables) are reusable values in CSS:

```
/* Without @property (basic) */
:root {
  --primary-color: #2196F3;
  --spacing-unit: 8px;
}

/* Using @property (advanced) */
@property --primary-color {
  syntax: '<color>';
  initial-value: #2196F3;
  inherits: true;
}
```

The basic approach works, but `@property` adds validation and features!

2.2 Traditional CSS Variables

```
:root {
  --color: blue;
  --size: 16px;
  --opacity: 0.5;
}

/* Use anywhere */
.box {
  background: var(--color);
  font-size: var(--size);
  opacity: var(--opacity);
}
```

Simple but limited. `@property` solves this!

3 @property Syntax

3.1 Basic Syntax

```
@property --my-property {  
  syntax: '<type>';  
  initial-value: <value>;  
  inherits: true | false;  
}
```

3.1.1 Three Required Descriptors

Descriptor	Purpose	Example
syntax	Property type	'<color>'
initial-value	Default value	blue
inherits	Inheritance	true or false

3.2 Supported Syntax Types

Syntax	Type	Example
'<color>'	Color	blue, #2196F3
'<length>'	Length	10px, 2em
'<length-percentage>'	Length or %	50%, 20px
'<number>'	Number	1, 0.5
'<percentage>'	Percentage	50%, 100%
'<angle>'	Angle	45deg, 1rad
'<time>'	Time	1s, 500ms
'<string>'	Text	'hello'
'<url>'	URL	url(image.jpg)
'*'	Any value	anything

4 Color Properties

4.1 Color Property Example

```
@property --primary-color {
  syntax: '<color>';
  initial-value: #2196F3;
  inherits: true;
}

:root {
  --primary-color: #2196F3;
}

.button {
  background: var(--primary-color);
  color: white;
}
```

4.2 Multiple Color Variables

```
@property --primary {
  syntax: '<color>';
  initial-value: #2196F3;
  inherits: true;
}

@property --accent {
  syntax: '<color>';
  initial-value: #FF5722;
  inherits: true;
}

@property --background {
  syntax: '<color>';
  initial-value: #FFFFFF;
  inherits: false; /* Don't inherit */
}

:root {
  --primary: #2196F3;
  --accent: #FF5722;
  --background: white;
}
```


5 Numeric Properties

5.1 Length Properties

```
@property --spacing-unit {
  syntax: '<length>';
  initial-value: 8px;
  inherits: false;
}

@property --border-radius {
  syntax: '<length>';
  initial-value: 4px;
  inherits: false;
}

:root {
  --spacing-unit: 8px;
  --border-radius: 4px;
}

.card {
  padding: calc(var(--spacing-unit) * 3);
  border-radius: var(--border-radius);
}
```

5.2 Percentage Properties

```
@property --opacity-level {
  syntax: '<percentage>';
  initial-value: 100%;
  inherits: true;
}

@property --width-ratio {
  syntax: '<length-percentage>';
  initial-value: 100%;
  inherits: false;
}

.element {
  opacity: calc(var(--opacity-level) / 100);
  width: var(--width-ratio);
}
```

5.3 Number Properties

```
@property --z-index-base {
  syntax: '<number>';
  initial-value: 1;
  inherits: false;
}

@property --line-height-multiplier {
  syntax: '<number>';
  initial-value: 1.5;
  inherits: true;
}

.modal {
  z-index: calc(var(--z-index-base) * 100);
}

body {
  line-height: var(--line-height-multiplier);
}
```

6 Animated Properties

6.1 Animating Custom Properties

The main advantage of `@property` is smooth animations!

```
@property --gradient-angle {
  syntax: '<angle>';
  initial-value: 0deg;
  inherits: false;
}

.gradient-box {
  background: linear-gradient(
    var(--gradient-angle),
    red,
    blue
  );
  animation: rotateGradient 3s infinite;
}

@keyframes rotateGradient {
  0% {
    --gradient-angle: 0deg;
  }
  100% {
    --gradient-angle: 360deg;
  }
}
```

6.2 Color Animation

```
@property --theme-color {
  syntax: '<color>';
  initial-value: #2196F3;
  inherits: true;
}

.button {
  background: var(--theme-color);
  transition: --theme-color 0.3s ease;
}

.button:hover {
  --theme-color: #1976D2;
}

/* Smooth color transition! */
```

7 Real-World Examples

7.1 Example 1: Design System with @property

```
/* Colors */
@property --primary {
  syntax: '<color>';
  initial-value: #2196F3;
  inherits: true;
}

@property --secondary {
  syntax: '<color>';
  initial-value: #FF5722;
  inherits: true;
}

/* Spacing */
@property --spacing-xs {
  syntax: '<length>';
  initial-value: 4px;
  inherits: false;
}

@property --spacing-sm {
  syntax: '<length>';
  initial-value: 8px;
  inherits: false;
}

@property --spacing-md {
  syntax: '<length>';
  initial-value: 16px;
  inherits: false;
}

/* Border Radius */
@property --radius-sm {
  syntax: '<length>';
  initial-value: 4px;
  inherits: false;
}

:root {
  --primary: #2196F3;
  --secondary: #FF5722;
  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
```

```
--radius-sm: 4px;
}
```

7.2 Example 2: Theme Switcher

```
@property --text-color {
  syntax: '<color>';
  initial-value: #000000;
  inherits: true;
}

@property --bg-color {
  syntax: '<color>';
  initial-value: #FFFFFF;
  inherits: false;
}

:root {
  --text-color: #000000;
  --bg-color: #FFFFFF;
}

html.dark-mode {
  --text-color: #FFFFFF;
  --bg-color: #1E1E1E;
}

body {
  background: var(--bg-color);
  color: var(--text-color);
  transition: background 0.3s, color 0.3s;
}
```

7.3 Example 3: Animated Gradient

```
@property --grad-angle {
  syntax: '<angle>';
  initial-value: 0deg;
  inherits: false;
}

@property --grad-color-1 {
  syntax: '<color>';
  initial-value: #FF6B6B;
  inherits: false;
}

@property --grad-color-2 {
  syntax: '<color>';
  initial-value: #4ECDC4;
  inherits: false;
}

.animated-gradient {
```

```

    background: linear-gradient(
      var(--grad-angle),
      var(--grad-color-1),
      var(--grad-color-2)
    );
    animation: gradientShift 6s infinite;
  }

  @keyframes gradientShift {
    0% {
      --grad-angle: 0deg;
    }
    100% {
      --grad-angle: 360deg;
    }
  }
}

```

7.4 Example 4: Responsive Spacing

```

@property --base-size {
  syntax: '<length>';
  initial-value: 16px;
  inherits: false;
}

:root {
  --base-size: 16px;
}

@media (max-width: 768px) {
  :root {
    --base-size: 12px;
  }
}

.container {
  padding: var(--base-size);
}

.heading {
  font-size: calc(var(--base-size) * 2);
}

.text {
  font-size: var(--base-size);
}

/* All scale together! */

```

8 Advanced Techniques

8.1 Computed Values

```
@property --hue {
  syntax: '<number>';
  initial-value: 0;
  inherits: false;
}

.box {
  background: hsl(
    var(--hue),
    100%,
    50%
  );
  transition: --hue 0.5s;
}

.box:hover {
  --hue: 120; /* Green */
}
```

8.2 Responsive Properties

```
@property --font-size-hero {
  syntax: '<length>';
  initial-value: 72px;
  inherits: false;
}

@property --font-size-body {
  syntax: '<length>';
  initial-value: 16px;
  inherits: true;
}

:root {
  --font-size-hero: 72px;
  --font-size-body: 16px;
}

@media (max-width: 768px) {
  :root {
    --font-size-hero: 48px;
    --font-size-body: 14px;
  }
}
```

```
}  
  
h1 {  
  font-size: var(--font-size-hero);  
}  
  
body {  
  font-size: var(--font-size-body);  
}
```

8.3 Fallback Values

```
@property --primary-color {  
  syntax: '<color>';  
  initial-value: #2196F3;  
  inherits: true;  
}  
  
/* With fallback */  
.button {  
  background: var(--primary-color, blue);  
}
```


9 Browser Support

9.1 Compatibility

Browser	Support
Chrome	85+
Edge	85+
Firefox	128+ (experimental)
Safari	16.4+
Opera	71+

9.2 Progressive Enhancement

```
/* Fallback for unsupported browsers */
.box {
  background: blue;
  background: var(--primary-color, blue);
}

/* This works even without @property */
```

10 Common Patterns

10.1 Pattern 1: Design Tokens

```
/* Color tokens */
@property --color-primary { syntax: '<color>'; initial-value: #2196F3;
    inherits: true; }
@property --color-secondary { syntax: '<color>'; initial-value: #FF5722;
    inherits: true; }
@property --color-danger { syntax: '<color>'; initial-value: #F44336;
    inherits: true; }

/* Typography tokens */
@property --font-size-sm { syntax: '<length>'; initial-value: 12px;
    inherits: true; }
@property --font-size-md { syntax: '<length>'; initial-value: 16px;
    inherits: true; }
@property --font-size-lg { syntax: '<length>'; initial-value: 24px;
    inherits: true; }
```

10.2 Pattern 2: State Management

```
@property --is-active {
    syntax: '<number>';
    initial-value: 0;
    inherits: false;
}

.toggle {
    --is-active: 0;
    background: gray;
}

.toggle.active {
    --is-active: 1;
    background: green;
}
```

10.3 Pattern 3: Animations Library

```
@property --animation-duration {
    syntax: '<time>';
    initial-value: 0.3s;
    inherits: false;
}
```

```
@property --animation-timing {  
  syntax: '<string>';  
  initial-value: 'ease';  
  inherits: false;  
}  
  
.fade-in {  
  animation: fadeIn var(--animation-duration) var(--animation-timing);  
}  
  
@keyframes fadeIn {  
  from { opacity: 0; }  
  to { opacity: 1; }  
}
```

11 Common Mistakes

11.1 Mistake 1: Missing Required Descriptors

DON'T:

```
@property --color {  
  syntax: '<color>';  
  /* Missing initial-value and inherits! */  
}
```

DO:

```
@property --color {  
  syntax: '<color>';  
  initial-value: blue;  
  inherits: true;  
}
```

11.2 Mistake 2: Wrong Initial Value Type

DON'T:

```
@property --size {  
  syntax: '<length>';  
  initial-value: 'medium'; /* String, not length! */  
  inherits: false;  
}
```

DO:

```
@property --size {  
  syntax: '<length>';  
  initial-value: 16px; /* Valid length */  
  inherits: false;  
}
```

11.3 Mistake 3: Forgetting Inheritance

PROBLEM:

```
@property --text-color {  
  syntax: '<color>';  
  initial-value: black;  
  inherits: false; /* Child doesn't inherit! */  
}
```

SOLUTION:

```
@property --text-color {  
  syntax: '<color>';  
  initial-value: black;  
  inherits: true; /* Child inherits color */  
}
```

12 Quick Reference

12.1 @property Syntax

```
@property --my-property {  
  syntax: '<color>' | '<length>' | '<percentage>'  
    | '<number>' | '<angle>' | '<time>' | '<string>' | '*';  
  initial-value: <value>;  
  inherits: true | false;  
}
```

12.2 Common Properties

```
/* Colors */  
@property --primary-color { syntax: '<color>'; initial-value: blue;  
  inherits: true; }  
  
/* Sizes */  
@property --spacing { syntax: '<length>'; initial-value: 8px; inherits:  
  false; }  
  
/* Numbers */  
@property --opacity { syntax: '<number>'; initial-value: 1; inherits:  
  false; }  
  
/* Angles */  
@property --rotation { syntax: '<angle>'; initial-value: 0deg; inherits:  
  false; }
```

12.3 Key Points

- Define types with `syntax`
- Set defaults with `initial-value`
- Control inheritance with `inherits`
- Animate custom properties smoothly
- Use for design tokens and themes
- Progressive enhancement works well
- Browser support is good (85)

Conclusion

CSS `@property` is a powerful way to create type-safe, well-documented custom properties. While custom properties (CSS variables) work without it, `@property` takes them to the next level with validation, animations, and better tooling support.

Key Takeaways:

- `@property` adds type safety to custom properties
- Use for design tokens and theme systems
- Enable smooth animations on custom properties
- Three required descriptors: `syntax`, `initial-value`, `inherits`
- Progressive enhancement keeps it safe
- Modern browsers support it well

Start using `@property` to build more robust, maintainable CSS systems!